

2.4 - Microserviços e bounded contexts

Princípios antes da tecnologia

O que define um microsserviço

Quatro princípios inegociáveis

- **Coesão em torno de um único domínio de negócio**
 - O serviço faz uma coisa e a faz por inteiro
- **Propriedade exclusiva dos próprios dados**
 - Nenhum outro serviço acessa o banco diretamente
- **Implantável de forma totalmente independente**
 - *Sobe, escala e falha sem depender dos vizinhos*
- **Comunicação apenas por contratos explícitos**
 - A fronteira do serviço é a sua interface pública

Bounded context: dentro vs. fora

Dentro da fronteira

- Modelo de domínio válido e consistente
- Linguagem ubíqua compartilhada pelo time
- Regras de negócio coesas e isoladas
- Mudanças internas não vazam para fora

Fora da fronteira

- Outro modelo para o mesmo termo de negócio
- Integração só por contrato, nunca por banco
- Acoplamento mínimo e sempre explícito
- Cada contexto evolui no seu próprio ritmo

Onde termina o tracking e começa a rota?

- **Tracking de motoristas tem escala e latência próprias**
 - Time decide extrair o primeiro serviço do monólito

LUNALOG

A LunaLog decidiu extrair o primeiro serviço: o tracking de motoristas, com requisitos de latência e escala bem diferentes do resto. No whiteboard, parecia trivial. Na prática, virou uma semana de debate. Onde termina o tracking e começa a gestão de rotas? O evento de localização do motorista pertence a qual serviço? A equipe descobriu que desenhar o limite certo era o trabalho de verdade, e que bounded context era exatamente a ferramenta que faltava.

Identificar limites na prática é difícil

- **O limite errado gera serviços que mudam juntos**
 - O acoplamento alto reaparece disfarçado de distribuição
- **Comece pelas fronteiras naturais do domínio**
 - Siga a linguagem que o negócio já usa
- **Dados que mudam juntos pertencem ao mesmo serviço**
 - *Coesão de dados é uma pista forte de fronteira*
- **Prefira poucos serviços bem definidos a muitos frágeis**
 - É mais fácil dividir depois do que juntar de novo

A fronteira é a decisão, não a tecnologia

- A tecnologia do serviço é o detalhe mais fácil
- O limite de domínio é a decisão mais cara

Microserviços falham quando o foco vai para frameworks e infraestrutura antes de a fronteira de domínio estar clara. Acerte o bounded context e o resto é implementação.

- Um limite mal traçado custa meses para corrigir



Limites de serviço definidos. Isso é só metade do trabalho. Fazer esses serviços conversarem de forma confiável é um problema completamente diferente, e é onde a maioria dos projetos tropeça. Como os serviços se comunicam, e quem é o dono de cada dado quando ninguém compartilha banco?

Comunicação, contratos e propriedade de dados